

General formulation of optimal Control Problems.

$\dot{x}(t) = f(x(t), u(t), t)$ under $x(t_0), x(t_f)$, admissible $x(t)$ & $u(t)$ constraints.

Find x^* & u^* s.t. $\min J = h(x_f, t) + \int_{t_0}^{t_f} g(x(t), u(t), t) dt$

- soln. x^* & u^* not guaranteed to exist OR be unique.

If $u^* = \pi(x(t), t) \rightarrow$ closed-loop opt. policy

else $u^* = e(x(t_0), t) \rightarrow$ open-loop opt. policy: only suits a given $x(t_0)$

NOC: $\nabla f(x^*) = 0$ (First-order) SOC: $\nabla f(x^*) = 0$
 $\nabla^2 f(x^*) \succ 0$ (2nd-order) $\nabla^2 f(x^*) \succeq 0$

Special OCs for convex functions:

f is convex if $f(\alpha x + (1-\alpha)y) \leq \alpha f(x) + (1-\alpha)f(y)$

- if f is convex, local min = glob. min

- if f is convex, $\nabla f(x^*) = 0$ is the only N&SOC.

INDIR. METHODS: up first Calculus of variations.

(closest thing to analytical soln. to developing controller).

Idea is to learn dynamics first. - then solve for opt. π^*

CoV 1-liner: The Euler Eqn. $\frac{\partial g}{\partial x}(x^*, u^*, t) - \frac{d}{dt} \frac{\partial g}{\partial u}(x^*, u^*, t) = 0$

where $x^*(t)$ is a proper extrema for the functional

$J(x) = \int_{t_0}^{t_f} g(x(t), \dot{x}(t), t) dt$

KEY ASSUMPTIONS to Euler Eqn: $g \in C^2$ (twice differentiable)
 to, t_f , x_0 , x_f are FIXED

KEY PROPERTIES of Euler Eqn: nonlinear, ordinary, 2nd-order

The VARIATIONAL APPR. Generalized Hamiltonian & PMP.

Problem Statement: with sys $\dot{x}(t) = f(x(t), u(t), t)$ Find adim. u^*
 that follows an adim x^* & minimizes $J(u) = h(x_f, t_f) + \int_{t_0}^{t_f} g(x, u, t) dt$

Approach: FORM GEN. HAMILTONIAN.

(note: p is the n-dim costate vector)

$\dot{x}^*(t) = \frac{\partial H}{\partial p}(x^*, u^*, p^*, t)$

$\dot{p}^*(t) = -\frac{\partial H}{\partial x}(x^*, u^*, p^*, t)$

$0 = \frac{\partial H}{\partial u}(x^*, u^*, p^*, t)$

$0 = \frac{\partial H}{\partial u}(x^*, u^*, p^*, t)$

$0 = \frac{\partial H}{\partial u}(x^*, u^*, p^*, t)$

$0 = \frac{\partial H}{\partial u}(x^*, u^*, p^*, t)$

$0 = \frac{\partial H}{\partial u}(x^*, u^*, p^*, t)$

$0 = \frac{\partial H}{\partial u}(x^*, u^*, p^*, t)$

$0 = \frac{\partial H}{\partial u}(x^*, u^*, p^*, t)$

$0 = \frac{\partial H}{\partial u}(x^*, u^*, p^*, t)$

$0 = \frac{\partial H}{\partial u}(x^*, u^*, p^*, t)$

$0 = \frac{\partial H}{\partial u}(x^*, u^*, p^*, t)$

$0 = \frac{\partial H}{\partial u}(x^*, u^*, p^*, t)$

$0 = \frac{\partial H}{\partial u}(x^*, u^*, p^*, t)$

$0 = \frac{\partial H}{\partial u}(x^*, u^*, p^*, t)$

$0 = \frac{\partial H}{\partial u}(x^*, u^*, p^*, t)$

$0 = \frac{\partial H}{\partial u}(x^*, u^*, p^*, t)$

$0 = \frac{\partial H}{\partial u}(x^*, u^*, p^*, t)$

$0 = \frac{\partial H}{\partial u}(x^*, u^*, p^*, t)$

BUT highlighted term assumes unconstrained u ! Hence PMP

$u^*(t) = \arg \min_u H(x, u, p, t)$

is the cond. that replaces

* PMP problems likely lead to bang-bang controls.

* a long-ass BC term for $x_f(t_f)$

if applicable (not written)

CLOSED LOOP METHODS $\rightarrow$ DP, LQR, iLQR, SDP, $\rightarrow$ not covered, similar to DP

Dynamic Programming: Built upon the PRINC. of OPTIMALITY

(under a Markovian structure)

P.O.: if $a \rightarrow b \rightarrow e$ is optimal path, tail $b \rightarrow e$ must be as well.

DP is great for small problems & doesn't need gradients or calc,

BUT curse of dimensionality $\rightarrow$ explodes in higher-order state spaces.

DP + LQR problem $\leadsto$ DTRE (Riccati eqn). input: classic LQR

DTRE: $P_k = Q + A^T P_{k+1} A - A^T P_{k+1} B (R + B^T P_{k+1} B)^{-1} B^T P_{k+1} A$

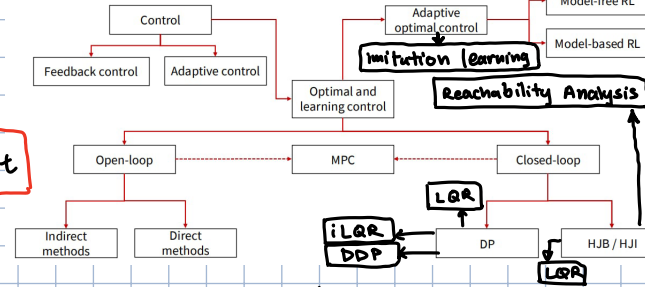
where you start at P_N = terminal cost & work BACKWARDS to P_0 .

OUTPUTS: $K_k = (R + B^T P_{k+1} B)^{-1} B^T P_{k+1} A$ * where K_k

is the opt. policy $u_k = K_k x_k$ at discrete-time k !

iLQR: LQR but for nonlinear, nonquadratic systems & costs.

The idea: LQR in-the-loop of SCP approximations at each k



Math Reminder: $\succ$ & $\succeq$

$\forall x: x^T A x \geq 0$

Then $A \succeq 0$

A is symmetric by definition

Some grad. desc. options

DIR: $d^k = -D^k \nabla f(x^k)$

$D^k = I$: steepest desc.

$D^k = (\nabla^2 f(x^k))^{-1}$: Newton's Method

with Ineq. Constraints.

KKT NOCs:

$L(x, \lambda, \mu) := f(x) + \sum_{i=1}^m \lambda_i h_i(x) + \sum_{j=1}^n \mu_j g_j(x)$

then x^* is opt. when there exists unique $(\lambda^*, \dots, \lambda_m^*)$ & $(\mu_1^*, \dots, \mu_n^*)$ s.t.

$\nabla_x L(x^*, \lambda^*, \mu^*) = 0$

$\mu_j^* \geq 0$ for active constr.

$\mu_j^* = 0$ for inactive constr.

Optimization Theory

Classic gradient descent

- key question:

selecting descent direction & step size

STEP:

- greedy minimizing

- const. - decaying.

Constrained optimization.

with Equality constraints

let $x \in \mathbb{R}^m$ with $f(x)$

s.t. $h_1(x) \dots h_n(x) = 0$

where $n < m$ or no soln exists.

Formulate the Lagrangian function

$L(x, \lambda) = f(x) + \sum_{i=1}^m \lambda_i h_i(x)$

NOCs: $\nabla_x L(x^*, \lambda^*) = 0$

$\nabla_\lambda L(x^*, \lambda^*) = 0$

Applications of PMP & the

classic "shifty controllers"

that result:

Min-time prob: $J = \int_{t_0}^{t_f} 1 dt \rightarrow$ Bang-Bang Ctrl

$u \in \{u_{\max}, u_{\min}\}$

Min-fuel prob: $J = \int_{t_0}^{t_f} C(u(t)) dt \rightarrow$ Bang-OFF-Bang Ctrl.

$u \in \{u_{\max}, 0, u_{\min}\}$

Min-energy p.: $J = \frac{1}{2} \int_{t_0}^{t_f} u^T R u dt \rightarrow$ Saturating Ctrl.

$u \in (u_{\min}, u_{\max})$

DIRECT METHODS (transcription from OCP $\rightarrow$ NLP)

collocation: parameterize $x_i, u_i \rightarrow \min_{x_i, u_i} \sum h_g(x_i, u_i)$

- Larger NLP, but structured

- more robust & stable

- Better for long-horiz. or stiff problems.

shooting: parameterize u_i only $\rightarrow \min_{u_i} \sum h_g(x_i, u_i)$

- smaller NLP

- Easier to implement $\rightarrow$ BUT more sensitive to inits.

SCP (sibling 2) you have system with nonlinear dynamics $f(\cdot)$

① Linearize around (x_k, u_k) using 1st order Taylor

$f_{k+1} = f_k + \frac{\partial f_k}{\partial x} (x - x_k) + \frac{\partial f_k}{\partial u} (u - u_k)$

② use transcription methods to turn continuous

LOCP $\rightarrow$ DLOCP ③ SOLVE.

HJI reachability & HJB. - introduce the idea of an "adversarial"

environment. Best to approach HJI with a 2-agent game of tag in mind.

HJI finds SAFE policy under WORST CASE scenarios (worst disturbances d)

$0 = \frac{\partial J}{\partial t} + \max_u \min_d [g(x, u, d) + \frac{\partial J}{\partial x} \cdot f(x, u, d)]$

with $u^* = \arg \max_u \min_d [g(x, u, d) + \frac{\partial J}{\partial x} \cdot f(x, u, d)]$

*NOTE! this is your WORRIER method.

HJB: The "no ops" guy (a non-existent)

$$0 = \frac{\partial J}{\partial t} + \min_u \left[g(x,u,t) + \frac{\partial J}{\partial x} \cdot f(x,u,t) \right]$$

with u^* = also just repl. $\min_u$ w argmin

REACHABILITY ANALYSIS

① Avoidance
Inputs: system + unsafe veg
Outputs: u^* , BRS of all states leading to Danger.

③ Reaching
Inputs: system + goal
Outputs: u^* , BRS of GOAL.

BRS: All states that can reach (or avoid) target set under time T .
BAT: The BRS across all times t . (it is a hyper tube).

Avoidance HJI: $\frac{\partial J}{\partial t} + \max_u \min_d \left[\frac{\partial J}{\partial x} f(x,u,d) \right]$

Reaching HJI: $\frac{\partial J}{\partial t} + \min_u \max_d \left[\frac{\partial J}{\partial x} f(x,u,d) \right]$

* reach/avoid are the same problem formed as a maximin or a minimax.

MPC: Open + Closed Loop! Closed Loop step: take only the 1st output of open-loop step.

EACH STEP solves a N-finite time OCP!

Formulation of finite problem @ each step.
 $J^*(x) = \min_{u_t \dots u_{t+N}} \left[P(x_{t+N}) + \sum_{k=0}^{N-1} c(x_{t+k}, u_{t+k}) \right]$
subj to $x_{t+k+1} = Ax_{t+k} + Bu_{t+k} \quad k=0, \dots, N-1$
 $x_{t+k} \in X \quad x_{t+N} \in X_f$
(x admiss.) (init) (u admiss.)

$P(x_{t+N})$: state-cost function (usually $l-1, l-2$, or $l-\infty$ norm)
 $x_N \in X_f$: X_f is the terminal set that must BE FEASIBLE.

Persistent - feasibility issue: don't go somewhere where no feasible N-step rollouts exist.
Stability Issue: ensure trajectories reach target.

* BOXED TERMS ARE KEY MPC DESIGN PTS.

① Control - Invariant set. [& maximal] **C**: if $C \subseteq X$ & for system $x_{t+k} = \phi(x_t, u_t)$, $x_t \in X$, $u_t \in U$, where if $x(t) \in C \Rightarrow \exists u \in U$ s.t. $\phi(x, u) \in C$. "once in C, it's always POSSIBLE to stay in C."
C ties neatly into the Persistent feasibility lemma $\rightarrow X_f$ must be control-invariant. **HUGE**

To ADDRESS stability issues: use the Lyapunov Thm.

Overview of learning: using "measurements" (maybe rollouts) to improve performance.

Measurements to improve controller:

- direct adaptive control $\rightarrow$ one-epis. **ONLINE**
- imitation learning $\rightarrow$
- model-free RL $\rightarrow$ multi-epis. **ONLINE**

Measurements to improve model:

- system identification $\rightarrow$ zero-epis. **OFFLINE**
- indirect adaptive control $\rightarrow$ one-epis. **ONLINE**
- model-based RL $\rightarrow$ multi-epis. **ONLINE**

System ID $\rightarrow$ relies on engineer-in-loop to drive a data-driven (often linear regression) model.
Math reminder: soln. to least-squares:
$$\hat{\theta} = (\mathbf{Z}^T \mathbf{Z})^{-1} \mathbf{Z}^T \mathbf{y}$$

Persistent excitation: $\hat{\theta} = (\mathbf{Z}^T \mathbf{Z})^{-1} \mathbf{Z}^T \mathbf{y} = (\mathbf{Z}^T \mathbf{Z})^{-1} \mathbf{Z}^T \mathbf{E} + \mathbf{0}$
@ each step you get $E[\hat{\theta}] = \mathbf{0}$ & $\text{cov}[\hat{\theta}] = \sigma^2 \frac{1}{N} \begin{bmatrix} x_1^2 & x_1 u_1 & u_1^2 \\ x_2^2 & x_2 u_2 & u_2^2 \\ \vdots & \vdots & \vdots \end{bmatrix}$... and you re-take until convergence.

Adaptive Control Lyapunov theorem for stability (stability is $\heartsuit$, optimality can wait)
consider autonomous system $\dot{x} = f(x, u = \pi)$
Equil. point @ $f(x^*) = 0$.
Then to be Lyapunov stable, some fn. $V(x)$ must exist where $V(x) > 0$, $V(x) < 0$, $V(x) \rightarrow \infty$ as $\|x\| \rightarrow \infty$ (for all $x \neq x^*$)

MRAC vs MIAC $\rightarrow$ REVIEW IF TIME ALLOWS.

RL elements: policy $\pi(u|x)$, env. $p(x_{t+1}|x, u)$, reward $R(x, u)$, value function $V(x) \rightarrow$ places value on state only
Model (optional $\ddot{o}$) Q -function $Q(x, a) \rightarrow$ places value on state-action pair.

Deeper IL dive 2 weaknesses: **Compounding Error**: simple errors build up
Paradox of IL: it might be better to train on data containing mistakes.
DAgger (Dataset Aggregation $\rightarrow$ engi-in-the-loop method of IL)
X weakness: querying an "expert" can be expensive.

AA228 speed review
 $\pi^* = \arg \max_{\pi} E \left[\sum_t \gamma^t R(x_t, \pi(x_t)) \right]$
policy maximizes cum. disc. reward
 $V_{\pi}(x_t) = E \left[\sum_{t'} \gamma^{t'-t} R(x_{t'}, \pi(x_{t'})) \right]$
 $Q_{\pi}(x_t, u_t) = E \left[\sum_{t'} \gamma^{t'-t} R(x_{t'}, u_{t'}) \right]$

MODEL-FREE RL Basis: TD Learning Update
$$V(s)_{\text{new}} = V(s)_{\text{old}} + \alpha (r + \gamma V(s') - V(s))$$

(TD error)
which merges MC & DP by learning from experience & bootstrapping on old estimates.
SARSA $\rightarrow$ on-policy π M-F RL.
Q-learning $\rightarrow$ OFF-policy M-F RL.
parameters. $\rightarrow$ use θ & α machinery of shit together (NN) to estimate $V_{\theta}(s)$

What if no access to dynamics? (AKA T is a mystery)
REINFORCE - "don't learn $V(x)$, just learn $\pi_{\theta}(a|s)$ parameterized by θ using NN."
- high variance by θ using NN.
- update by episode
 $\theta \leftarrow \theta + \alpha \nabla_{\theta} \log \pi(a|s; \theta) R$
Actor-Critic - Actor moves $\pi(a|s; \theta)$ critic moves $V(s; w) \dots$
- lower variance
- more stability
A2C: $A(s, a) = Q(s, a) - V(s)$ (subflavor)